02/09/2026

Level 3 : Recursion

① Find Sum of array using Recursion

Problem: Given, arr = [2, 4, 6, 8, 10] find the sum recursively 2+4+6+8+10 = 30

Recursion thinking : Sum(n) = n + Sum(n-1) → same i/p

array: Sum(arr, index) = Current element + Sum(remaining elements)

Sum(arr, 0) = arr[0] + Sum(arr, 1).

Python code:

```
def sum_arr(arr, index):
    if index == len(arr):
        return 0
    return arr[index] + sum_arr(arr, index+1)
```

arr = [2, 4, 6, 8, 10] o/p : 30

T.C : O(n)
S.C : O(n)

Rule:

Array
↓
Index
↓
Current Element
+
Recursive rest

② Reverse a string using Recursion

Problem: Given a string, reverse it using recursion

i/p: "hello" o/p: "olleh"

Recognition keywords: Reverse a string, Reverse a character, print string backwards, Recursive reverse, reverse without

Pattern: Solve the smaller string
↓
Add current character

Formula: reverse(s) = reverse(s[1:]) + s[0]

Array Recursion:

arr[index] + solve(index + 1)

String Recursion:

solve(s[1:]) + s[0]

Python Template:

```
def reverse_string(s):
    if len(s) <= 1:
        return s
    return reverse_string(s[1:]) + s[0]
s = "hello"
print(reverse_string(s))
```

"abc":

reverse("bc") + "a"

reverse("c") + "b" + "a"

→ "cba"

T.C: O(n²) → slicing & concatenation

S.C: O(n)

③ Print a linked list using Recursion

problem: Given 1→2→3→4→None using recursion 1 2 3 4 print.

Recognition keywords: Recursive linkedlist traversal, print linkedlist recursively, process every node recursively

Python template:

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

def print_list(head):
    if head is None:
        return
    print(head.val)
    print_list(head.next)

def print_reverse(head):
    if head is None:
        return
    print_reverse(head.next)
    print(head.val)
```

T.C : O(n)
S.C : O(n)

Pattern:

Work → Recursive = Forward

Recursive → Work = Reverse

(A) Binary Search using Recursion

problem: Given a sorted array and a target recursively find the target's index.

i/p : arr = [1, 3, 5, 7, 9, 11] } o/p : 3
target = 7

Recognition keywords: Sorted array, Find target, Search in sorted array, O(log n) search, Binary Search using recursion

Pattern:

Find mid
↓
arr[mid] == target?
↓
Yes → Return mid
↓
No
→ target < arr[mid] → left half
→ target > arr[mid] → right half

Python Template:

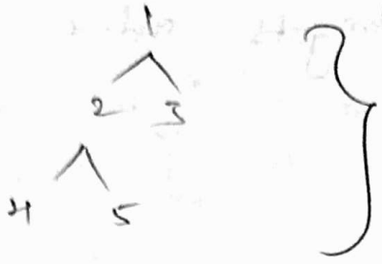
```
def binary_search(arr, target, left, right):
    if left > right:
        return -1
    mid = (left + right) // 2
    if arr[mid] == target:
        return mid
    if target < arr[mid]:
        return binary_search(arr, target, left, mid-1)
    return binary_search(arr, target, mid+1, right)
arr = [1, 3, 5, 7, 9, 11]
target = 7
print(binary_search(arr, target, 0, len(arr)-1))
```

o/p: 3

T.C: O(log n)
S.C: O(log n)

⑤ Tree traversal using Recursion

Problem: Given this binary tree:



using Recursion perform a inorder traversal.

O/P: 4 2 5 1 3

Concept: Node → Left, Right

Inorder pattern: Left → Root → Right

Solve(left) process(root) Solve(right)

Recognition words: Inorder traversal, Binary tree, traverse left and right subtree, Recursive tree traversal, print tree nodes.

Three important traversals:

Preorder → Root → Left → Right

Inorder → Left → Root → Right

postorder → Left → Right → Root

Python Template:

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

def inorder(root):
    if root is None:
        return
    inorder(root.left)
    print(root.val)
    inorder(root.right)
```

T.C: O(n)

S.C: O(h)

~ Recursion completed ~